

DevOps Engineering Assignment

AWS Cloud Migration — Infrastructure as Code

Individual Project • 2-Week Timeline

Overview

You are tasked with designing and implementing an AWS cloud migration for a microservices platform currently running on-premise. This is an individual assignment — you are expected to understand every decision you make. AI tools may be used as aids, but you will be tested on your reasoning.

Background & Requirements

A company runs 8 microservices (Java/Node.js) on-premise with an Nginx load balancer, PostgreSQL database (500 GB), and needs to migrate to AWS. Key requirements:

- 99.99% uptime SLA with automatic failover
- Auto-scaling from 1,000 to 5,000 req/sec during peak traffic
- 50,000 concurrent users across Europe and US
- Container-based deployments (Docker) on EC2
- Full CI/CD pipeline integrated with GitHub
- Observability: metrics, logs, and alerting

Stage 1 — Architecture Design

Before writing a single line of code, produce a clear architecture document. This stage is the foundation — a weak architecture means a failed project.

Deliverable: Architecture Document

Create an architecture. It must include:

- AWS Services Diagram — draw or describe all components and how they connect
 - VPC layout: public/private subnets, availability zones (minimum 2 AZs)
 - Load balancing: Application Load Balancer (ALB) in front of EC2 instances
 - EC2 Auto Scaling Groups for each microservice
 - RDS PostgreSQL (Multi-AZ) replacing on-premise DB
 - Supporting services: S3, CloudWatch, IAM roles
- Design Decisions — for each major choice, explain WHY
 - Why EC2 over ECS/Lambda for this workload?
 - Why Multi-AZ RDS? What does it protect against?

- How does the architecture achieve 99.99% uptime?
- Network & Security Model
 - Which resources live in public vs private subnets?
 - Security groups: what ports, what sources?
 - How are secrets stored and accessed by services?
- Scaling Strategy — how does the system handle 1K to 5K req/sec?

Stage 2 — GitHub Repository Setup

Create a private GitHub repository that will host all project artifacts. Structure matters — a well-organized repo communicates professionalism and maintainability.

Repository Structure

Your repo must follow this layout:

```
my-aws-migration/
├── architecture.png          # Stage 1 deliverable
├── README.md                 # Setup and deployment guide
├── terraform/               # All infrastructure code
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── modules/             # VPC, EC2, RDS, ALB, IAM
└── .github/
    ├── workflows/
    │   ├── ci.yml           # Build & test pipeline
    │   └── cd.yml           # Deployment workflow (no apply)
```

README Requirements

- Project overview and architecture summary
- Prerequisites (AWS CLI, Terraform, GitHub Actions setup)
- Step-by-step: how to run terraform plan locally
- How to trigger the CI pipeline

Stage 3 — CI/CD Pipeline (GitHub Actions)

Implement GitHub Actions workflows that automate testing and validate infrastructure. All pipelines run in GitHub — no manual steps required beyond a git push.

Workflow 1: CI Pipeline (ci.yml)

Triggers on every push to any branch and on all pull requests to main.

- Lint & Validate — fail fast on obvious errors
 - terraform fmt -check — formatting validation
 - terraform validate — configuration syntax check
 - tflint — Terraform best practices linter
- Security Scan
 - tfsec or checkov — scan for hardcoded secrets, open security groups, unencrypted resources
- Terraform Plan
 - Run terraform plan and output the plan as a PR comment
 - Use GitHub OIDC (not hardcoded keys) to authenticate with AWS
 - Plan only — never run terraform apply in CI

Workflow 2: Deploy Workflow (cd.yml)

Triggers only on merge to main branch. Runs terraform plan and saves the output as a GitHub Actions artifact for review. Never executes apply.

- Run terraform init and terraform plan -out=tfplan
- Upload the plan file as a workflow artifact
- Post a summary to the GitHub Actions job summary page

Stage 4 — Terraform Infrastructure Code

Write the Terraform code that defines the full AWS architecture. You must be able to run terraform plan successfully in your local environment. Terraform apply is NOT required.

Required Resources

- Networking (terraform/modules/vpc/)
 - VPC with CIDR block, 2 public + 2 private subnets across 2 AZs
 - Internet Gateway, NAT Gateway, route tables
- Compute (terraform/modules/ec2/)
 - Launch Template with user data to install Docker and run microservice containers
 - Auto Scaling Group: min=2, max=10, target CPU utilization policy - bonus, if exists you will be asked about it
 - Application Load Balancer with target groups and health checks

- Database (terraform/modules/rds/)
 - RDS PostgreSQL, Multi-AZ enabled, encrypted at rest
 - Automated backups, 7-day retention
 - DB subnet group in private subnets only
- Security (terraform/modules/iam/)
 - IAM roles and instance profiles for EC2
 - Security groups — principle of least privilege
 - Secrets Manager resource for DB credentials
- Observability
 - CloudWatch Log Groups for each microservice
 - CloudWatch Alarms: CPU > 80%, ALB 5xx errors > 1%

Terraform Standards

- Use variables.tf for all configurable values — no hardcoded values in resources
- Use outputs.tf to expose key resource IDs (ALB DNS, RDS endpoint, VPC ID)
- Remote backend: S3 + DynamoDB state locking (configured, not applied)
- All resources must have Name and Environment tags

Evaluation Criteria

You will present your work and be asked to explain your decisions. Understanding matters as much as completion.

Important Notes

- Work alone — you will be evaluated individually.
- AI tools are permitted as aids. You must be able to explain everything in your submission.
- Do NOT run terraform apply — terraform plan output is sufficient.
- All secrets must use GitHub Actions secrets or AWS Secrets Manager — never committed to code.
- Your CI pipeline must pass (green) before submission.